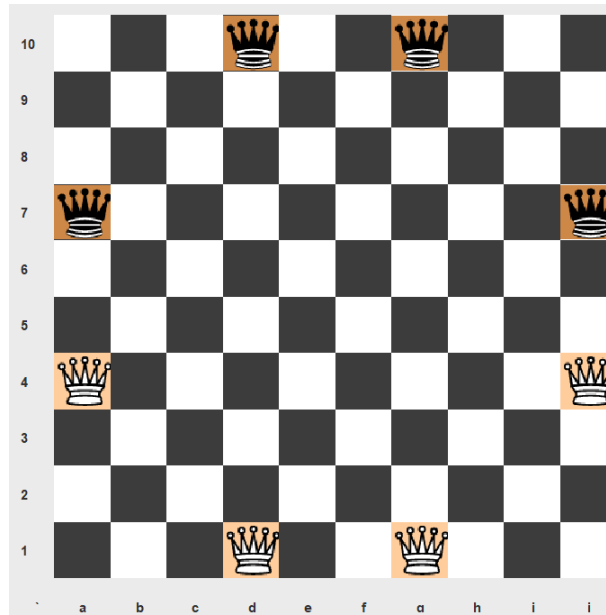


COSC 322 Project Report

Harsimar Singh, Gurbir Amrit, Lakshay Karnwal, Vanshika, Osho Gnanasivam

Introduction- Game of Amazons



The Game of Amazons is a dual-player strategic board game. It consists of a 10x10 grid where players have four amazons each at their fixed starting positions. The objective of this game is to block the way of an opponent's amazons using arrows thereby making some squares impassable according to the game rules. To make a play, the player will move one Amazon and then take a shot from their new location. The game stops when a player can no longer move legally; the player with the most mobility wins.

Our Goal

We decided to implement the AI bot in the Game of Amazon with a minimax search and Alpha-Beta pruning to improve its decision-making capability for choosing the best move. The bot infers the current state of the board and tries to predict moves that would put our opponent at a disadvantage while, at the same time, benefiting our side. It does this by simulating possible board states from legal moves from a given position and then evaluating a heuristic score of this position.. Since the complete number of moves when analyzing a single

state of the game board can be a large number, we use Alpha-Beta pruning to reduce the number of suboptimal branches that our algorithm searches when analyzing the game tree of moves. Our bot will attempt to place arrows and produce and move the amazons in such a way as to force the enemy's play space down to a single or few deadly spots, anticipating any moves the enemy bot might make and responding to them.

Techniques Used

Heuristic Evaluation of a Board State

In order to implement any type of decision-making algorithm, we first had to figure out a way of evaluating the favorability of a particular board position by assigning a numerical value. This numerical value is based on subjective factors that we believe are important to the progression of the game. The factors and how we calculated a value to estimate its prominence are as follows:

- **Territory Control:** This factor is calculated by estimating the distances of each queen to unoccupied tiles on the board. We use Manhattan distances while taking obstructions (other pieces and arrows) into account to calculate these distances and use this distance to evaluate the “influence” of a player on the remaining empty tiles and the level of mobility that the pieces have.
- **Mobility:** Similar to territory control, we use distances and level of access of each queen to calculate an aggregate level of mobility that a player has based on the queen's positions in a given board state.
- **Centralization/Avoiding Borders:** Much like in games like Chess and Shogi, centralization of pieces is a vital strategy employed by professional players. A central piece will likely have more access to more parts of the board as it will have more directions to consider compared to a piece in the corner/border. As such, we place importance on centralization by increasing the weight of this factor when our pieces end up in the corners in certain states.
- **Avoiding Clustering of Queens:** Queens (of the same colour) that are densely packed together will not cover as many tiles on the board as there will likely be more overlap in the tiles they have access to. Moreover, they tend to obstruct each other and prevent mobility. As such, we penalize the score of states where our queens are clustered

together hoping to avoid these situations. Our goal is to spread out our pieces strategically across the board so that if the opponent is blocking one area, we don't risk multiple of our queens.

Other Techniques:

Distance Calculation: We are using the Breadth-First Search (BFS) algorithm to determine the distances between a specific piece and all other nodes in the graph representing the game board. These distances are then used to provide valuable information about the accessibility of squares and the potential reach of a player's moves.

Normalisation Factor: Based on the factors below, we calculate the weightage of each factor mentioned below and above, to figure out how much priority should be placed on each strategy at a given state. We add the factors below and above using the following formula:

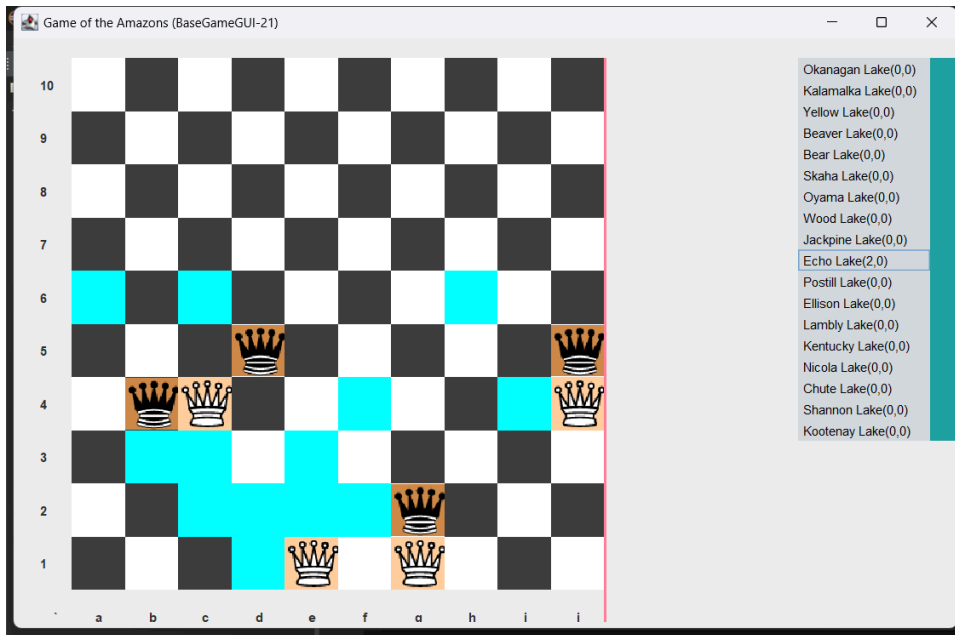
$$\sqrt{TerritoryScore^2 + MobilityScore^2 + ClusteringScore^2 + BorderingScore^2}$$

This value is used as the denominator of each score to establish the weight of consideration when calculating the total score of the board.

Dynamic Evaluation 1 and 2: Method 1 (Territory) is used to figure out the net influence of the white queen and the black queen on the game board. If the score is positive, it indicates that the white queen has more influence overall and if negative, the opposite is true. The more extreme the values, the more favourable it is to the colour (negative->black, positive->white). Method 2(Mobility) represents the difference in the distances of cells from the black queen and the white queen. If the score is negative, it indicates that unoccupied tiles are closer to the black queen on average and vice-versa.

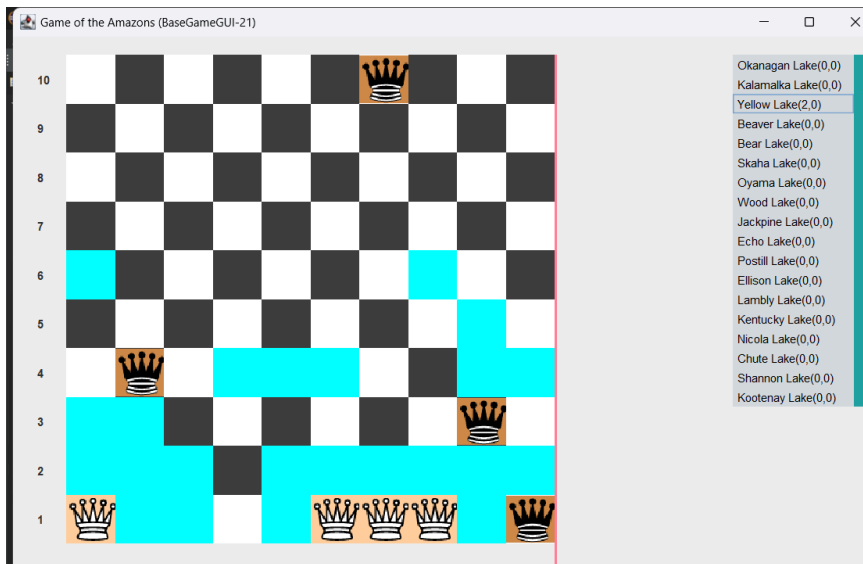
We also used a few other methods to assist in making decisions such as which queen to move based on distances to other queens.

EXAMPLES - GAME 1 : Bot 15 vs another bot (Middle state)



BOT 15 is playing black and we can see how we are cornering all the opponent queens one by one. The average heuristic score for this state is -23, which means black is favoured and more chances for us to win.

GAME 2: Bot 15 vs another Bot (final state)



Bot 15 is playing black and wins. Have a border avoidance score of 0.35, having control over most of the board. The heuristic score for the bot is -58, favouring our bot the most.

Game Tree Search: MinMax with Alpha-Beta Pruning

Our implementation focuses on creating a tree using the potential game states to evaluate the best possible move. In this approach, the player that aims to maximize their score is called the maximizer, and the player that aims to minimize the maximizer's score is the minimizer. Next, the algorithm recursively explores the game tree at each depth level using the Depth First Algorithm. At each level, if it is the maximizer's turn, the algorithm chooses the move that leads to the child node with the highest score, if it is the minimizer's turn, it chooses the move that leads to the child node with the lowest node. In our code, we have set the depth threshold to 6, which means a maximum of 189700 nodes (derived from trial and error) will be explored within the time limit of 28 seconds. Once the exploration reaches any of these threshold values (6 for depth, 189700 for nodes, 28 seconds for time), the tree returns the current best move it finds.

One common challenge with using the Min-Max algorithm is that as the game complexity increases, the number of nodes to be explored grows, making the computation very extensive. To counter this, we ensure that the depth we search is reduced depending on the size of legal moves from a given state available for exploration using the following algorithm:

$$\text{if } \text{NumberOfLegalMoves}^{\text{depth}} > 189700, \text{depth} - -$$

We also used Alpha-Beta Pruning to overcome this challenge. This pruning technique removes irrelevant or unnecessary branches of the search tree. Alpha represents the highest score the maximizer that is possible at a certain depth, and Beta represents the lowest value the minimizer can limit the maximizer to at that depth. The Min-Max algorithm, along with the Alpha-Beta pruning, produces the optimal move for the playing player hence giving the best chance of winning against the opponent.

Classes Description

The project is divided into three major sections:

- Tree
- Algorithm
- Game Coordination

Tree

The Tree section includes the Tree, TreeNode and ActionsGenerator classes

These classes represent the game board as a Tree. Every tile on the game board represents a node (TreeNode), and the connection between two nodes is represented using TreeEdge.

Key methods:

- `updateTreeWithNewAction(ActionsGenerator.Action currentMove, Tile currentPlayer)`: updates the Tree State with the new move made by the player
- `possibleMoves(Tree gameState, Tile activePlayer)`: This is a static method that calculates all possible moves a player can make from the current game state.

Algorithm

This section is used for the decision-making aspect of the code. It includes

HeuristicEvaluation and AlphaBetaMiniMax and a few nested classes for evaluating the best move.

The heuristic score evaluations take place inside the HeuristicEvaluation class. Then, the AlphaBetaMiniMax class is used to traverse the tree using the MinMax algorithm to evaluate the best possible move.

Key Methods:

- `heuristicCalculator(Tree gameState, ubc.cosc322.Tile currentPlayer)`: computes the aggregated (territory control, board dominance, mobility, piece clustering) heuristic score for a game state for a specified player.
- `optimalActionCalculator(Tree gameboard, Tile activePlayer, int depth, long timeBegin)`: This function finds the optimal move for a player based on the current game state, then initiates alpha-beta pruning. The Alpha-beta pruning receives specific search depth and execution timing values.
- `ABprune(long timeBegin, Tree gameboard, int depthSearch, float A, float B, Tile activePlayer, ActionsGenerator.Action oldAction)`: This function uses alpha-beta pruning, to explore the game tree. The Alpha-beta pruning only occurs until the set depth gets rid of unnecessary branches

- AlphaBetaMiniMax.PrunedResult: Encapsulates the Alpha-Beta pruning outcome, including a move (ActionsGenerator.Action) and its associated heuristic value. Higher values denote more favorable outcomes, reflecting the move's efficacy from the current player's perspective.

Game Coordination

The Game Coordination part includes the GameCoordinator Bot15 classes. These classes coordinate the game state, move generation, and the decision-making process. The Bot15 class extends the GamePlayer class to play as the AI.

Key methods:

- updateBoardState(ActionsGenerator.Action recievedMove): based on the given move, updates the legal moves map and the current board state; it then returns a map with the details of the move.
- bestMove(long startTime): this method finds the optimal move for the current player by calling the optimal move methods from the HeuristicEval

Conclusion

The heuristics implemented worked well in our favour for the most part of the tournament. Territory control and mobility score are fairly intuitive heuristics to consider as part of our strategy while border avoidance and clustering was a strategy we adopted by playing the game manually and learning from it. Although these techniques helped us play very well defensively, we were definitely lacking strategy in terms of being more aggressive and attempting to immobilise the opponent's queens. We learned that the team we had lost to had implemented heuristics based on particular stages of the game (starting, middle and endgame), indicating that different strategies may need to be adopted based on the phase of the game much like in Chess and Shogi. Techniques to figure out phases of the game also requires more human gameplay experience and we hope to implement it some day in the future.